CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Overview

This is a **ROS1 (Catkin) workspace** for LiDAR-based robotic spraying path planning. It processes RSLidar point clouds to detect geometric targets (cylinders or I-beams/工字钢), computes slicing grids on their surfaces, and publishes target poses for a robotic arm sprayer.

Build & Run

```
# Build
cd ~/projects/ros_airy_work
catkin_make
source devel/setup.bash

# Launch cylinder fitting pipeline
roslaunch pcl_process cylinder_fitting.launch

# Launch I-beam fitting pipeline
roslaunch pcl_process ibeam_fitting.launch

# Manual node startup
rosrun pcl_process rslidar_process_node_ros    # LiDAR filter node
rosrun pcl_process cylinder_fitting_node_ros   # Cylinder detection node
rosrun pcl_process ibeam_fitting_node_ros      # I-beam detection node
```

Trigger detection via topic:

```
rostopic pub /achieve_work_pose std_msgs/Bool "data: true"
```

Publish process parameters at runtime:

```
rostopic pub /process_param_bridge pcl_process/ProcessParam \
  "distance: 750  step_width: 300  step_height: 100  height: 600  lap: 0.5
arm_reach: 1500  mode: 2  k: 0  w0: 0"
```

Tune detection parameters without recompiling:

```
# Edit YAML then re-launch, or use rqt_reconfigure at runtime
rqt_reconfigure
```

Offline PCD analysis:

```
cd src/pcl_process/src/cylinder_fitting
python3 cylinder_yaml_den.py <file.pcd> [--tilt-angle 45]  # generates
recommended_cylinder_params.yaml

cd src/pcl_process/src/cal_logs
python3 statics.py 12-22/res1 [--viz] [--compare 12-22/res2]
```

Package Structure

```
src/
├── main/               # Thin entry package (package.xml only, no active
nodes currently)
└── pcl_process/        # Main processing package
    ├── msg/            # Custom ROS messages
    ├── config/         # YAML parameter files (edit to tune without
recompiling)
    │   ├── cylinder_detection_params.yaml
    │   └── ibeam_detection_params.yaml
    ├── launch/
    │   ├── cylinder_fitting.launch
    │   └── ibeam_fitting.launch
    └── src/
        ├── rslidar_process/    # RSLidarFilterNode: filters raw cloud into
two topics
        ├── cylinder_fitting/   # CylinderFitter + CylinderFittingROS nodes
        ├── ibeam_fitting/      # IBeamFitter + IBeamFittingROS nodes
        ├── tf/                 # FlexedArmTF: hardcoded LiDAR→Base
transform
        ├── pcd_viewer/         # Offline PCD viewer utility
        ├── tests/              # Test utilities
        └── cal_logs/           # Log analysis tools (statics.py) and
interface docs
```

Architecture

Data Flow

```
RSLidar → /rslidar_points
            ↓ RSLidarFilterNode (rslidar_process)
            ├─→ /rslidar_filtered_cylinder  (±40° sector)
            └─→ /rslidar_filtered_ibeam     (±50° sector)
                        ↓
        ┌──────────────────────────────────┐
        ↓                                  ↓
CylinderFittingROS                IBeamFittingROS
```

```
        ↓                                   ↓
/target_pose_list              /target_pose_list
/spray_target_info             /ibeam_target_info
/spray_info                    /spray_info
/process_param                 /ibeam_process_param
```

Coordinate Frames

All frames managed by `FlexedArmTF` (`src/pcl_process/src/tf/lidar2flexed_arm.hpp`):

- **O** – Robot base frame (X-forward, Y-left, Z-up). All output target poses are in this frame.
- **B** – LiDAR frame (Y-forward, X-right, Z-up). Raw point cloud input.
- **C** – Small arm frame (aligned with O, offset from O). Used for some work-point calculations.
- Transform is **hardcoded** (not from ROS TF tree): LiDAR offset from arm end-effector A = (820, 0, -160) mm; A in base = (590, 2470, 2560) mm with +90° yaw.
- All internal calculations in **mm**; `/target_pose_list` output is in **meters**.
- Z-axis is lifted by `MIN_WORK_HEIGHT_MM = 2000 mm` when converting to base frame.

Cylinder Detection Pipeline (`cylinder_fitting/`)

1. **ROI filter** – angle/distance pass-through on input cloud
2. **Wall removal** (`removeWalls()`) – iterative RANSAC plane removal (up to `max_walls` planes)
3. **Normal estimation** – KNN-PCA after wall removal
4. **RANSAC cylinder fit** (`SACSegmentationFromNormals`) – constrained by `ransac_axis_{x,y,z}` and `eps_angle_deg`
5. **Validation** – checks inlier count, ratio, height, radius range, radial std-dev
6. **Slicing** – `sliceCylinder()` generates `SliceData` objects (surface center, axis center, work point)
7. **Mode dispatch**:
   - **mode=2 (Auto)**: computes virtual base position, max coverage angle via cosine rule, optimizes overlap ratio
   - **mode=1 (Manual)**: uses user-supplied `k` and `lap` to compute slice width
8. **Publish** – sorted top-down target poses on `/target_pose_list`

I-Beam Detection Pipeline (`ibeam_fitting/`)

Uses DBSCAN clustering + PCA shape extraction. Outputs plane equations, axis vector, wing plate width, and surface center point. Refer to `src/pcl_process/src/ibeam_fitting/ibeam_fitting_node_issue.md` for known issues.

Dynamic Reconfigure

Both detection nodes support `dynamic_reconfigure`. Configs are in:

- `cfg/CylinderDetection.cfg` → `pcl_process/CylinderDetectionConfig`
- `cfg/IBeamDetection.cfg` → `pcl_process/IBeamDetectionConfig`

Parameters can be changed at runtime via `rqt_reconfigure` without restarting nodes.

Key Topics

Topic	Type	Direction	Description
`/rslidar_points`	sensor_msgs/PointCloud2	In	Raw LiDAR data
`/rslidar_filtered_cylinder`	sensor_msgs/PointCloud2	Internal	±40° filtered cloud
`/rslidar_filtered_ibeam`	sensor_msgs/PointCloud2	Internal	±50° filtered cloud
`/achieve_work_pose`	std_msgs/Bool	In	Start/stop detection trigger
`/work_object`	std_msgs/String	In	Target object type selector
`/process_param_bridge`	pcl_process/ProcessParam	In	Spraying process parameters
`/target_pose_list`	pcl_process/TargetPoseList	Out	Work points in base frame (m)
`/spray_target_info`	pcl_process/TargetInfo	Out	Detailed slice info (mm)
`/spray_info`	pcl_process/SprayInfo	Out	Total slice count and area
`/ibeam_process_param_bridge`	pcl_process/IBeamProcessParam	In	I-beam process parameters
`/ibeam_target_info`	pcl_process/IBeamTargetInfo	Out	I-beam detection result

Custom Messages (`msg/`)

- **ProcessParam** – spraying process config (distance, step_width, height, lap, arm_reach, mode, k, w0)
- **TargetPose** – single work point (id, x, y, z in meters, base frame)
- **TargetPoseList** – list of TargetPose
- **TargetInfo** – full slice geometry for debugging (circle_center, spray_center, axis, radius, angle, height, all in mm)
- **SprayInfo** – aggregate stats (num slices, total area m^2)
- **IBeamProcessParam / IBeamTargetInfo** – I-beam equivalents

Cylinder Detection Parameters

Key params in `config/cylinder_detection_params.yaml` (two profiles: vertical vs tilted cylinder):

Parameter	Vertical	Tilted	Description
`ransac_axis_{x,y,z}`	0,0,1	0.707,0,0.707	RANSAC reference axis

Parameter	Vertical	Tilted	Description
`eps_angle_deg`	15°	20°	Max axis deviation
`min/max_radius_m`	0.30–0.40	0.04–0.18	Target radius range
`use_ideal_axis`	true	false	Force vertical axis in slicing
`normal_distance_weight`	0.05	0.05	RANSAC normal weight

The YAML file contains commented-out tilted-cylinder configs. Swap sections when switching target types.

Notes

- The `src/main` package currently has no compiled nodes (just package.xml and CMakeLists.txt).
- `FlexedArmTF` offsets are **hardcoded hardware constants** — update them when physical arm geometry changes.
- `MIN_WORK_HEIGHT_MM = 2000mm` Z-lift is hardcoded in `cylinder_fitting_ros.cpp` — not in YAML.
- Cylinder validation thresholds (radius 0.30–0.40 m) are currently sized for one specific target. Adjust `cylinder_detection_params.yaml` for other targets.
- Log files are in `src/pcl_process/src/ibeam_fitting/log.txt` (gitignored).